DATABASE SYSTEM
FINAL PROJECT REPORT

Hospital Management System Using PHP and MySQL

Lecturer:
Mr. Ronny Juwono, S.pd.,M.T.

Informatics Class 1 Batch 2024

Group Members :

Cristine Valentina	001202400111
Naila Olivia Ramadhani	001202400112

PRESIDENT UNIVERSITY
Jababeka Education Park, Jl. Ki Hajar Dewantara, Kota Jababeka, Cikarang Baru, Bekasi 17550 – Indonesia

2025

PART A - DATABASE DESIGN CASE STUDY

A.1 Case Study Overview

The Hospital Management System is a helpful website created with PHP and MySQL to make hospital tasks much easier. Thij incfbbvv33ewawwzws system helps both patients and doctors by replacing paper records with digital ones.

Patients can use the system to sign up online, book doctor appointments, view their medical records, and pay their bills. At the same time, doctors can check their appointment schedules and access their patients' health information whenever needed.

This system makes hospital work simpler by safely storing all medical records in one place. This means less paperwork for staff and more time to focus on patient care. The website is easy to use for everyone, with strong security to keep private health information safe.

The Hospital Management System improves communication between patients and doctors by organizing important health information in ways that are easy to find. By handling routine tasks automatically, it helps doctors make better decisions, makes the hospital run more smoothly, and improves healthcare for everyone.

A.2 Business Functions

a. Patient
    1. patient register and login
        - a patient must have an account before they can access the website. they must fill the register form such as full name,username,password,address etc.
        - users have to login before they can enter the website.
    2. Book Appointment
        - patient can make appointment with doctor
        - the patient can select the doctor speciality
        - the patient can select a date.
    3. Find Doctor
        - patient can see the list of the doctor
        - patient can see doctor’s information
    4. Medical records
        - patient can see their medical records

b. Doctor
    1. Doctor register and login
        - a doctor must have an account before they can access the website. they must fill the register form such as full name,username,password,address etc.
        - doctors have to login before they can enter the website.
    2. Today's Appointment
        - doctor can see their schedule including time,patient,age/gender,status of the appointment
    3. Patient search
        - doctor can search their patient based on their information
    4. Upcoming Appointment
        - doctor can see upcoming appointment
    5. Schedule
        - doctor can see their schedule
    6. Medical Records
        - a doctor can make new record for patient
        - a doctor can put diagnosis and prescription
        - a doctor can delete medical records.

A.3 Data Requirement

1) Patient Entity:

- Patient ID (int (Primary Key, Auto Increment))
- Full Name (varchar (100))
- DOB (date)
- Gender (enum ('Male', 'Female', 'Other'))
- Phone (varchar (20)
- Email (varchar (100))
- Address (text)
- Username (varchar (50))
- Password (varchar (255))
- Created at (timestamp)

2) Doctor Entity:

- doctor id (int (Primary Key, Auto Increment))
- name (varchar (100))
- speciality id (int)
- room_number (varchar (20))
- phone (varchar (20))
- email (varchar (100))
- username (varchar (50))
- password (varchar (255))
- created at (timestamp)
- foreign key (speciality_id) references specialities (speciality_id)

3) Appointment Entity:

- appointment_id (int (Primary Key, Auto Increment))
- patient_id (int (11))
- doctor_id (int (11))
- appointment_datetime (datetime)
- status (enum ('Scheduled', 'Completed', 'Cancelled'))
- notes (text)

- created_at (timestamp)
- foreign key (patient_id) references patients (patient_id)
- foreign key (doctor_id) references doctors (doctor_id)

4) Medical Record Entity:

- record_id (int (Primary Key, Auto Increment))
- patient_id (int (11))
- doctor_id (int (11))
- appointment_id (int (11))
- diagnosis (text)
- prescription (text)
- notes (text)
- created_at (timestamp)
- foreign key (patient_id) references patients (patient_id)
- foreign key (doctor_id) references doctors (doctor_id)
- foreign key (appointment_id) references appointments (appointment_id)

5) Payment Entity:

- payment_id (int (Primary Key, Auto Increment))
- appointment_id (int (11))
- amount (decimal (10,2))
- payment date (date)
- payment_method (enum ('Cash', 'Credit Card', 'Insurance', 'Bank Tra..))
- status (enum ('Pending', 'Completed', 'Cancelled').
- foreign key (appointment_id) references appointments (appointment_id)

6) Specialty Entity:

- specialty_id (int (Primary Key, Auto Increment))
- name (varchar (100))

A.4. Business Rule

1. Before making an appointment, patients must register and log in.

2. User can log out from their account

3. Patient can book an Appointment

4. Patient will choose the doctor based on their speciality

5. Patient can select date and time for the appointment

6. Patient and doctor can see their scheduled appointment

7.. Only doctors who are available can be scheduled for appointments.

8. Only their own patients records may be viewed and updated by doctors.

9. Every appointment needs to be in one of the following valid states: pending, confirmed, completed, or cancelled.

10. The payment status needs to be noted as either paid or unpaid.

11.Only doctor who can edit delete and update medical records

Part B - Conceptual Data Modelling
B.1. Overview

The Hospital Edisur website is a PHP, MySQL web app that simplifies hospital tasks. Patients can register, book appointments, and view records, while doctors manage schedules and update health info. It reduces paperwork and improves care through secure, organized data access.

B.2. Revised Business Rules and Assumption of Part A

1. patients

- patient_id (Primary Key) : Unique identifier for each patient
- name (varchar) : Full name of the patient
- dob (date) : Date of birth
- gender (enum) : Gender of the patient ('Male', 'Female', 'Other')

- phone (varchar) : Patient's contact number
- email (varchar) : Patient's email address
- address (text) : Patient's home address
- username (varchar) : Patient's login username
- password (varchar) : Patient's hashed password
- created_at (timestamp) : Account creation timestamp

Assumptions:

- Each patient has a unique patient_id
- Required fields must be filled during registration
- Email and username must be unique
- Passwords are stored in hashed form
- created_at is auto-filled upon registration

2. doctors

- doctor_id (Primary Key) : Unique identifier for each doctor
- name (varchar) : Full name of the doctor
- speciality_id (int) : Foreign key referencing the doctor’s specialty
- room_number (varchar) : Doctor's consultation room
- phone (varchar) : Doctor's contact number
- email (varchar) : Doctor's email address
- username (varchar) : Doctor’s login username
- password (varchar) : Doctor's hashed password
- created_at (timestamp) : Account creation timestamp

Assumptions:

- Each doctor has a unique doctor_id
- Each doctor must be assigned to one specialty
- Email and username must be unique
- created_at records the doctor onboarding date
- Passwords are securely hashed

3. appointments

- appointment_id (Primary Key) : Unique identifier for each appointment
- patient_id (int) : Foreign key referencing the patient
- doctor_id (int) : Foreign key referencing the doctor
- appointment_datetime (datetime) : Date and time of the appointment
- status (enum) : Status of the appointment ('Scheduled', 'Completed', 'Cancelled')
- notes (text) : Notes written by the doctor
- created_at (timestamp) : Time when the appointment was created

Assumptions:

- Each appointment is uniquely identified
- Patients can only book available slots
- Status reflects the lifecycle of the appointment
- notes can be filled after consultation
- created_at is auto-filled on booking

4. medical_records

- record_id (Primary Key) : Unique identifier for each medical record
- patient_id (int) : Foreign key referencing the patient
- doctor_id (int) : Foreign key referencing the doctor
- appointment_id (int) : Foreign key referencing the related appointment
- diagnosis (text) : Doctor’s diagnosis
- prescription (text) : Medications or treatments prescribed
- notes (text) : Additional remarks from the doctor
- created_at (timestamp) : Time when the record was created

Assumptions:

- One medical record per appointment
- Records are created after the appointment is completed
- Only doctors can create or edit medical records
- created_at is filled upon record creation

5. payments

- payment_id (Primary Key) : Unique identifier for each payment
- appointment_id (int) : Foreign key referencing the appointment
- amount (decimal(10,2)) : Total amount to be paid
- payment_date (date) : Date of payment
- payment_method (enum) : Payment method ('Cash', 'Credit Card', 'Insurance', 'Bank Transfer')
- status (enum) : Status of the payment ('Pending', 'Completed', 'Cancelled')

Assumptions:

- Each payment is tied to a valid appointment
- One payment per appointment
- status indicates the current state of the transaction
- payment_method is chosen at time of transaction
- Amount includes consultation and other service charges

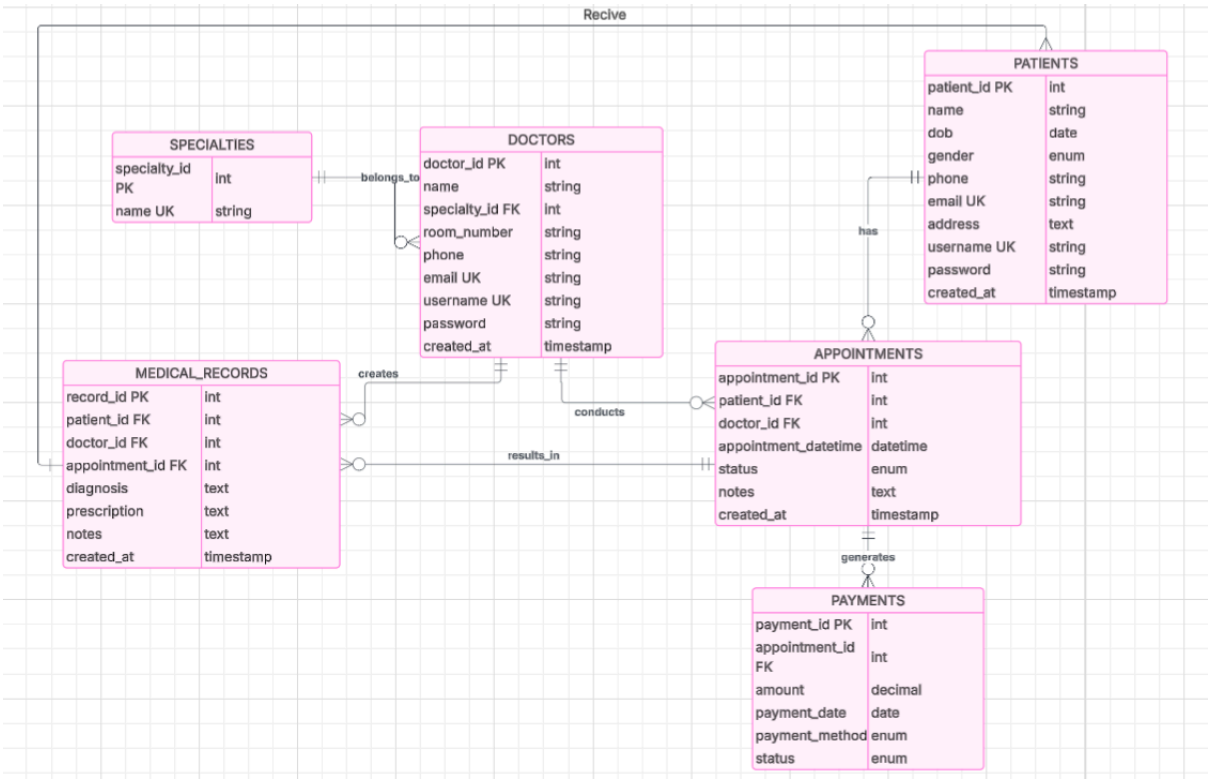
6. specialties

- specialty_id (Primary Key) : Unique identifier for each specialty
- name (varchar) : Name of the specialty (e.g., Cardiology, Pediatrics, Dermatology)

Assumptions:

- Each doctor is linked to one specialty
- Specialties are created by system admin or manager
- Specialty names must be unique to avoid confusion

B.3 ERD



B.4 Justify ERD Based on Business Rule

Entity: PATIENTS

Business Rules:

- BR1: Every patient must register with complete personal and login details to use the system.
- BR2: Each patient has a unique username and email.

Design Justification:

The PATIENTS entity includes fields such as patient_id (PK), name, dob, gender, phone, email, address, username, password, and created_at, fulfilling the requirement of capturing complete personal and login details.

Entity: DOCTORS

Business Rules:

- BR4: Each doctor must be assigned to one medical specialty.
- BR5: Doctors can have multiple appointments with different patients.
- BR11: Each doctor must have a secure login system with encrypted passwords.

Design Justification:
The DOCTORS entity contains attributes like doctor_id (PK), name, specialty_id (FK), room_number, phone, email, username, password, and created_at, allowing the assignment of specialties and supporting authentication and contact.

Entity: SPECIALTIES
Business Rules:

- BR12: Each specialty has a unique ID and name.

Design Justification:
The SPECIALTIES entity has specialty_id (PK) and name, which directly satisfy the need to manage medical specialties distinctly.

Entity: APPOINTMENTS
Business Rules:

- BR3: A patient can have multiple appointments, each associated with only one patient.
- BR6: An appointment must be linked to one doctor and one patient.
- BR7: Each appointment must have a status: Scheduled, Completed, or Cancelled.

Design Justification:
The APPOINTMENTS entity includes appointment_id (PK), patient_id (FK), doctor_id (FK), appointment_datetime, status, notes, and created_at, capturing the required scheduling and tracking data.

Entity: MEDICAL_RECORDS
Business Rules:

- BR8: Medical records are created after an appointment is completed.
- BR9: A medical record is uniquely associated with one appointment, one doctor, and one patient.

Design Justification:
The MEDICAL_RECORDS entity includes record_id (PK), patient_id (FK), doctor_id (FK), appointment_id (FK), diagnosis, prescription, notes, and created_at, linking medical details clearly to both the patient and the appointment.
Entity: PAYMENTS
Business Rules:

- BR10: Payments are made per appointment and can have different methods.

Design Justification:
The PAYMENTS entity includes payment_id (PK), appointment_id (FK), amount, payment_date, payment_method, and status, which supports the variety of payment methods and ensures tracking of financial transactions per appointment.

Part C - Logical design: Schema Conversion and Normalism
C.1 Revised Business Rules and Assumptions (From part B)

1. Patients must register with complete data before booking an appointment.
2. Doctors must be registered and assigned to one specialty before handling appointments.
3. Appointments must link both patient and doctor, and include a status: Scheduled, Completed, or Cancelled.
4. Medical records are created after completed appointments, with one record per appointment.
5. Only doctors can create or update medical records, including diagnosis and prescription.
6. Payments are valid only if linked to a completed appointment and must reflect status.
7. Each appointment generates one payment, covering all related charges.
8. Doctors are grouped by specialties, which are managed only by system admins.
9. Medical records and appointments are accessible only to authorized users.
10. Dashboard and booking access depends on login and user roles.
11. Doctor slideshow is filtered by specialty to enhance booking experience.
12. Patients can view medical records only from completed appointments with filled data.

C.2. Revised ERD of Part B

The ERD associated with our database Hospital Edisur case study remain unchanged from part B

C.3. Relations

We can convert the conceptual schema into relational tables.

- Patient Table
  Patient (patient_id, name, dob, gender, phone, email, address, username, password, created_at)

- Doctor Table
  Doctor (doctor_id, name, specialty_id*, room_number, phone, email, username, password, created_at)
  ForeignKey: specialty_id REFERENCES Specialty(specialty_id)

- Specialty Table
  Specialty (specialty_id, name)

- Appointment Table
  Appointment (appointment_id, patient_id*, doctor_id*, appointment_datetime, status, notes, created_at)
  ForeignKey: patient_id REFERENCES Patient(patient_id)
  ForeignKey: doctor_id REFERENCES Doctor(doctor_id)

- Medical_Record Table
  Medical_Record (record_id, patient_id*, doctor_id*, appointment_id*, diagnosis, prescription, notes, created_at)
  ForeignKey: patient_id REFERENCES Patient(patient_id)
  ForeignKey: doctor_id REFERENCES Doctor(doctor_id)
  ForeignKey: appointment_id REFERENCES Appointment(appointment_id)

- Payment Table
  Payment (payment_id, appointment_id*, amount, payment_date, payment_method, status)
  ForeignKey: appointment_id REFERENCES Appointment(appointment_id)

C.4. List of functional dependencies related to each business rule.

1. Patients (patient_id) → (name, dob, gender, phone, email, address, username, password, created_at)
2. Doctors (doctor_id) → (name, specialty_id, room_number, phone, email, username, password, created_at)
3. Specialties (specialty_id) → (name)
4. Appointments (appointment_id) → (patient_id, doctor_id, appointment_datetime, status, notes, created_at)
5. Appointments (patient_id) → (appointment_id)
6. Appointments (doctor_id) → (appointment_id)
7. Medical Records (record_id) → (patient_id, doctor_id, appointment_id, diagnosis, prescription, notes, created_at)
8. Medical Records (appointment_id) → (record_id)
9. Payments (payment_id) → (appointment_id, amount, payment_date, payment_method, status)
10. Payments (appointment_id) → (payment_id)
11. Doctors (specialty_id) → (doctor_id)
12. Medical Records (doctor_id) → (record_id)
13. Appointments (status) → (appointment_id)
14. Patients (email, username) → (patient_id)
15. Doctors (email, username) → (doctor_id)

C.5. Normalisation (Logical Design)

a. Functional Dependencies:

Patients Table

1. patient_id → name, dob, gender, phone, email, address, username, password, created_at
2. email → patient_id, name, dob, gender, phone, address, username, password, created_at
3. username → patient_id, name, dob, gender, phone, email, address, password, created_at
4. phone → patient_id, name, dob, gender, email, address, username, password, created_at
5. created_at → patient_id, name, dob, gender, phone, email, address, username, password

Doctors Table

1. doctor_id → name, specialty_id, room_number, phone, email, username, password, created_at
2. email → doctor_id, name, specialty_id, room_number, phone, username, password, created_at
3. username → doctor_id, name, specialty_id, room_number, phone, email, password, created_at
4. specialty_id → doctor_id, name, room_number, phone, email, username, password, created_at
5. created_at → doctor_id, name, specialty_id, room_number, phone, email, username, password

Specialties Table

1. specialty_id → name
2. name → specialty_id

Appointments Table

1. appointment_id → patient_id, doctor_id, appointment_datetime, status, notes, created_at

2. patient_id → appointment_id, doctor_id, appointment_datetime, status, notes, created_at
3. doctor_id → appointment_id, patient_id, appointment_datetime, status, notes, created_at
4. status → appointment_id, patient_id, doctor_id, appointment_datetime, notes, created_at

5. appointment_datetime → appointment_id, patient_id, doctor_id, status, notes, created_at
6. created_at → appointment_id, patient_id, doctor_id, appointment_datetime, status, notes

Medical Records Table

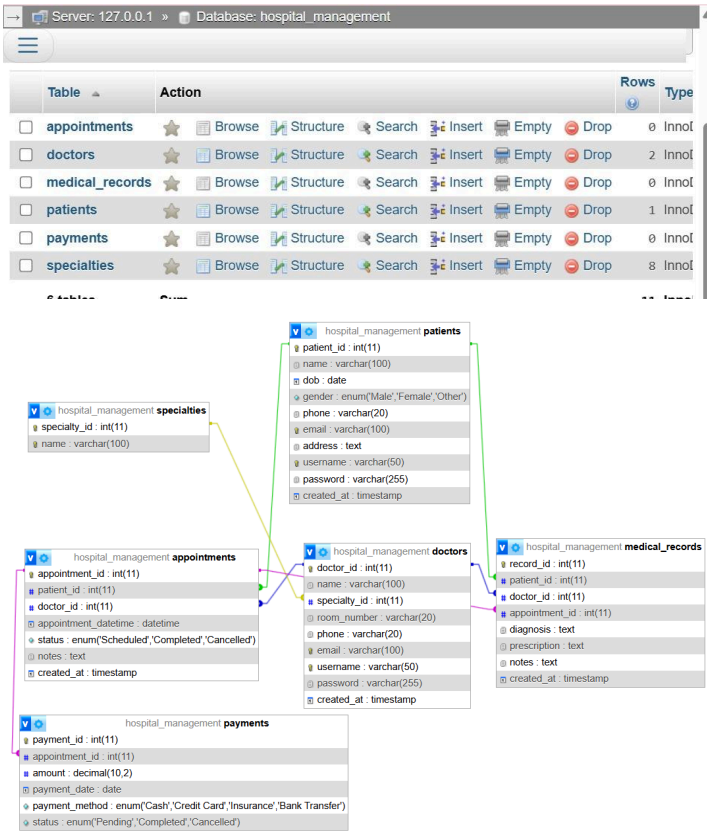
1. record_id → patient_id, doctor_id, appointment_id, diagnosis, prescription, notes, created_at
2. appointment_id → record_id, patient_id, doctor_id, diagnosis, prescription, notes, created_at
3. doctor_id → record_id, patient_id, appointment_id, diagnosis, prescription, notes, created_at
4. created_at → record_id, patient_id, doctor_id, appointment_id, diagnosis, prescription, notes

Payments Table

1. payment_id → appointment_id, amount, payment_date, payment_method, status
2. appointment_id → payment_id, amount, payment_date, payment_method, status
3. payment_date → payment_id, appointment_id, amount, payment_method, status
4. status → payment_id, appointment_id, amount, payment_date, payment_method

Part D - Implementation

D.1 The Database



D.2 The Data

- **The data of the appointment**

	appointment_id	patient_id	doctor_id	appointment_datetime	status	notes	created_at
Edit Copy Delete	1	1	9	2025-12-02 15:00:00	Scheduled		2025-04-15 15:22:22
Edit Copy Delete	2	2	28	2025-04-15 16:00:00	Scheduled		2025-04-15 16:11:45

- **The data of the doctors**

	doctor_id	name	specialty_id	room_number	phone	email	username	password	created_at
Edit Copy Delete	9	Andika Pratama	1	101A	081234567890	andika.pratama@hospital.com	drandika	$2y$10$2voHN3k48ZlHLjLZFhCS.efxlg0HR1Lu536Y2HqERv1...	2025-04-15 14:39:09
Edit Copy Delete	25	Siti Handayani	5	202B	0989385428	siti@gmail.com	drsiti	$2y$10$rcbsaed8O7GympSVIHRvcOBMG6Nj1Pd1SO6WpI4DbhK...	2025-04-15 16:00:04
Edit Copy Delete	26	Bambang Haryanto	8	303C	0834256272	bambang@gmail.com	drbang	$2y$10$Qmt1Ghc7GQRNG6TRezh8Ou29plUK1vRnKo3J2FxAgOil...	2025-04-15 16:02:49
Edit Copy Delete	27	Nina Paramitha	6	102A	08253563212	nina@gmail.com	drnina	$2y$10$fuxBDMLrx8QkwILQ51.fiOvDqrQppzMMQK/1.1jTj0a...	2025-04-15 16:04:07
Edit Copy Delete	28	Cristine Valentina	6	103A	08937492911	itin@gmail.com	dritin	$2y$10$VsC0OhGwjg4IA.5mCjLIl0iNdJdese5DbiiU2VuxM1D...	2025-04-15 16:05:11

- **The data of the medical record**

	record_id	patient_id	doctor_id	appointment_id	diagnosis	prescription	notes	created_at
Edit Copy Delete	1	2	28	2	ADHD			2025-04-15 16:54:37

Check all *With selected:* Edit Copy Delete Export

- **The data of the patients**

patient_id	name	dob	gender	phone	email	address	username	password	created_at
1	Naila Olivia Ramadhani	2005-10-10	Female	81384570236	naila.oliviaaa@gmail.com	perum.wahana pd gede blok m4 no2	fullsun	$2y$10$eYTBcl1sVV5I3HDGbs7xYOKcLLgMfzVVEC39hrDsIl6...	2025-04-14 15:19:54
2	Michelle	2006-01-02	Other	089823432234	misel@gmail.com	Medan	misel	$2y$10$0WKuBJT1tHWSXHg91yX82uYOZ9HSW6F3STEUL59uXw2...	2025-04-15 16:06:27

Check all With selected: Edit Copy Delete Export

- **The data of the payments**

SELECT * FROM `payments`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

payment_id	appointment_id	amount	payment_date	payment_method	status

Query results operations

Create view

- **The data of the specialities**

Extra options

specialty_id	name
1	Cardiology
5	Dermatology
8	ENT
6	Internal Medicine
2	Neurology
7	Ophthalmology
3	Orthopedics
4	Pediatrics

Check all With selected: Edit Copy Delete Export

D.3 The Queries

1. **Select Patient who have booked an appointment with specific doctor**

```
SELECT p.*
FROM patients p
JOIN appointments a ON p.patient_id = a.patient_id
WHERE a.doctor_id = 28;
```

patient_id	name	dob	gender	phor
2	Michelle	2006-01-02	Other	0898

2. **Select Doctor and their speciality**

```
SELECT
  s.name AS Specialty,
  d.name AS Doctor
FROM specialties s
JOIN doctors d ON s.specialty_id = d.specialty_id;
```

Specialty	Doctor
Cardiology	Andika Pratama
Dermatology	Siti Handayani
ENT	Bambang Haryanto
Internal Medicine	Nina Paramitha

3. **Group by How many Appointment each Doctor Has**

```
SELECT
  d.name AS Doctor,
  COUNT(a.appointment_id) AS total_appointments
FROM doctors d
JOIN appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.name;
```

Doctor	total_appointments
Andika Pratama	1
Cristine Valentina	1

4. **View Appointments and Patient Info**

```
SELECT
  p.name AS patient_name,
  COUNT(a.appointment_id) AS total_appointments
FROM patients p
JOIN appointments a ON p.patient_id = a.patient_id
GROUP BY p.patient_id
ORDER BY total_appointments DESC;
```

patient_name	total_appointments 1
Naila Olivia Ramadhani	1
Michelle	1

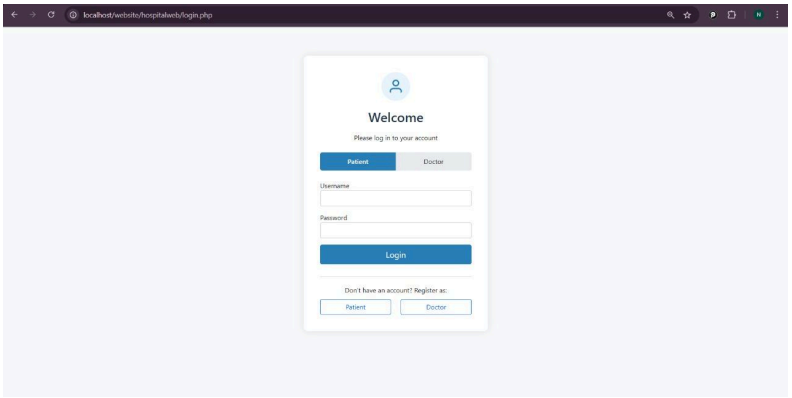
5. **List Upcoming Appointments (Future Dates)**

```
SELECT
  a.appointment_id,
  p.name AS patient_name,
  d.name AS doctor_name,
  a.appointment_datetime
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id
WHERE a.appointment_datetime > NOW()
ORDER BY a.appointment_datetime ASC;
```

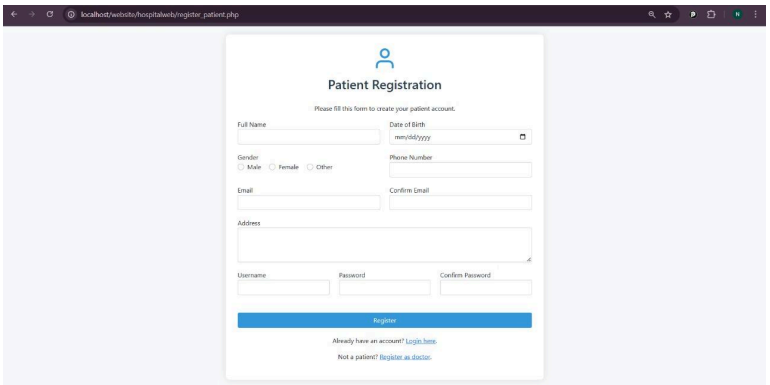
appointment_id	patient_name	doctor_name	appointment_datetime
1	Naila Olivia Ramadhani	Andika Pratama	2025-12-02 15:00:00

D.4 UI/UX and PHP-BASED DATABASE APPLICATION

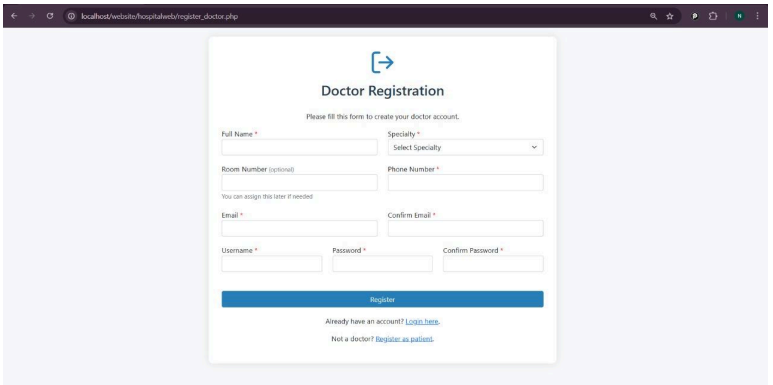
- **This is the login page**



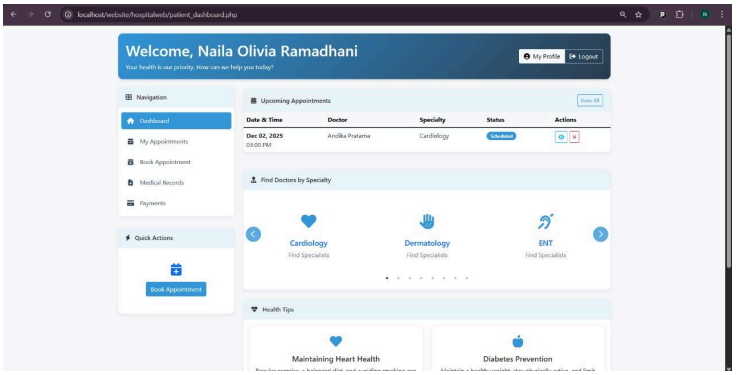
- **This is the registration for Patient**



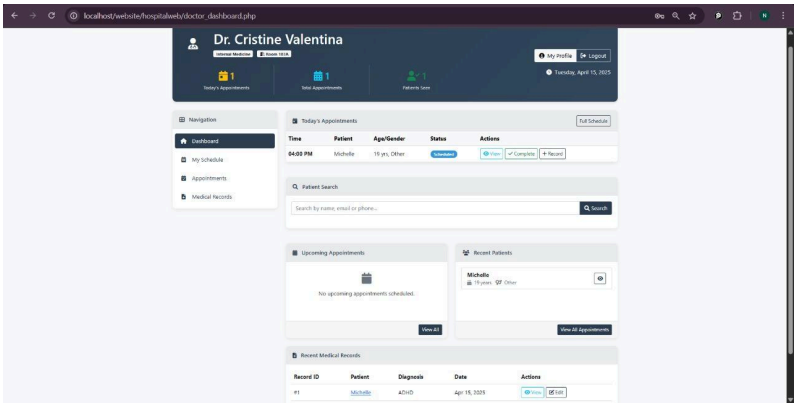
- **This is the registration for doctor**



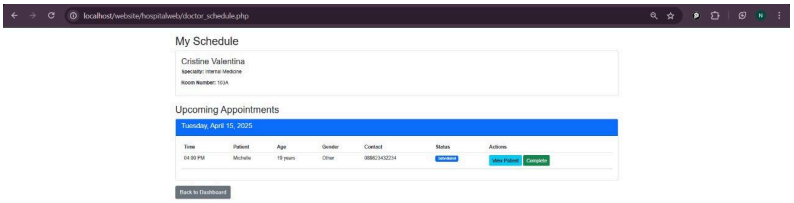
- **This is the dashboard patient**



- **This is the dashboard doctor**

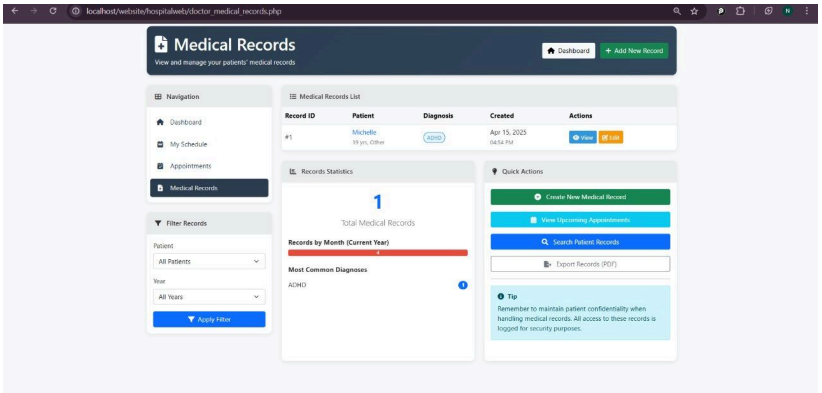


- **This is the doctor can see their schedule**

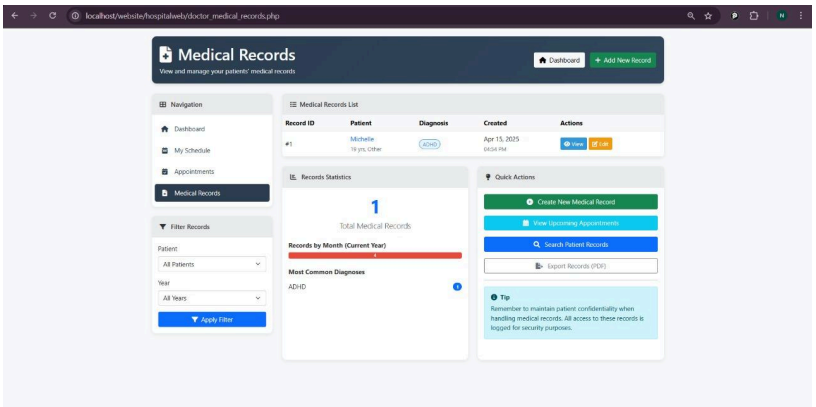
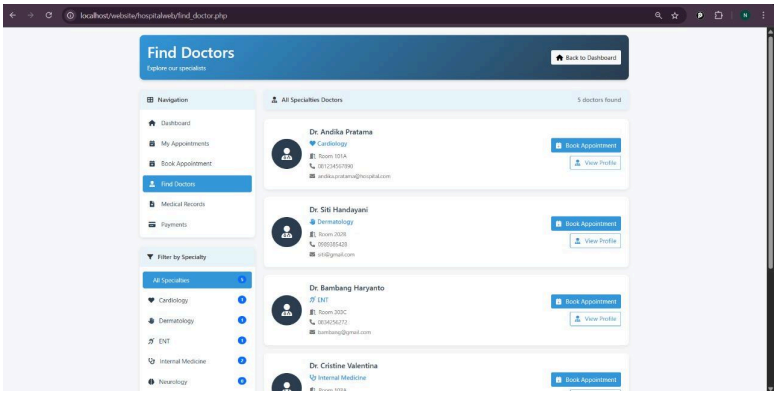


- **This is the doctor can see their appointment**

- **This is the Medical Record**



- **This is for add Medical Record Doctor**



- **This is for Find Doctors**